

LC-Cloud

Kubernetes 基盤 アーキテクチャ仕様書

OpenStack 5 ノード上の 2 クラスタ構成と、共用モデルによる Kubernetes 提供

対象	AS152865 / LC-Cloud
アンダーレイ	OpenStack 2026.1 — controller / compute / storage を全ノードで兼任
日付	2026-08-31

OVERVIEW

設計の骨子

#	項目	内容
1	クラスタ構成	platform (管理系) と shared-tenant (ユーザー系) の 2 クラスタ。分離の根拠はコントロールプレーン境界
2	Kubernetes 提供方式	クラスタ払い出しではなく、共用クラスタ上で利用者ごとに namespace を分離する
3	認証系	Authentik。server / worker / LDAP outpost と PostgreSQL で構成する
4	ノード役割	全 5 ノードが OpenStack の controller / compute / storage を兼任する
5	ストレージ	全 VM の root と追加ディスクは Ceph RBD。SSD の用途からの逸脱は etcd のみ
6	ネットワーク	Ceph 用ディスクネットワークを分離し jumbo で運用。WAN は IPoE の 1500。Geneve ヘッダ長 38 バイト、Pod MTU 1412
7	ロードバランサ	Octavia の ovn provider。amphora は使用しない。Platform が Gateway を独占する
8	テナント分離	Pod Security Admission の restricted、NetworkPolicy default-deny、gVisor を既定適用
9	SMT	SMT 側チャネルの残余リスクを受容する。スレッド分離は行わない
10	ノードの扱い	Kubernetes ノードは cattle として扱い、イメージ更新で入れ替える
11	BGP 収容	専用アプライアンスを持ち込まず、ソフトウェアルータで収容する
12	構成管理	Terraform が定義を生成し Git に置く。クラスタへの適用は Argo CD が担う

PREREQUISITES

ハードウェアとネットワークの前提

項目	内容
ノード	5 台。全ノード同一構成
CPU	Xeon Scalable Gen2。28 物理コア / 56 スレッド。SMT 有効
メモリ	128 GB / ノード
ローカル SSD	SATA SSD 1 TB / ノード。Power Loss Protection なし
Ceph OSD	HDD ×6 / ノード。replica 3。DB と WAL はローカル SSD 上
Ceph ディスクネットワーク	分離済み。jumbo frame で運用する
WAN	FLET'S IPoE。MTU 1500
オーバーレイ	Geneve。ヘッダ長 38 バイトで運用する
BGP 収容	OpenStack 上または OS 上のソフトウェアルータ
PLPを持たない SSD が制約となる理由	2026.1。kolla-ansible で構築する

BlueStore の DB と WAL は同期書き込みを行う。Power Loss Protection を持たない SSD ではキャッシュのフラッシュが書き込みごとに必要になるため、同期書き込み性能がエンタープライズ品に対して大幅に低下する。HDD OSD の遅延をローカル SSD で緩和する構成において、この緩和が期待どおり働かない可能性がある。

さらに、DB と WAL を置いたデバイスを失うと当該ノードの OSD がすべて同時に停止する。5 ノード replica 3 で 1 ノードが全損した場合の再構築は HDD 上で行われ、その期間は許容最小レプリカ数の下限で運転することになる。

この SSD は OS と Ceph の DB / WAL に加えて etcd も収容する。3 者が同一デバイスで同期書き込みを競合させるため、構築後に同期書き込み性能を実測し、専用 NVMe の追加要否を判断する。

NODE ROLES

ノード役割 — 全 5 ノードが 3 役を兼任する

サービス群	配置	制約
Nova compute / libvirt / ovn-controller	全 5 ノード	—
Ceph OSD	全 5 ノード	—
Ceph MON / MGR	3 または 5 ノード	MON は奇数。台数を決める必要がある
MariaDB (Galera)	3 ノード	奇数必須。このノードのメモリ消費が大きい
RabbitMQ	3 ノード	同上
OpenStack API 各サービス / Horizon	3 ノード	HAProxy と keepalived の VIP 配下
RGW	3 または 5 ノード	台数を決める必要がある
Kubernetes ノード	全 5 ノード	上記と同一ノード上の VM として動作する

設計上の帰結

第一に、ノード間が非対称になる。Galera・RabbitMQ・API 群を持つ 3 ノードと持たない 2 ノードでは VM に配れるメモリが異なるため、均等配置を前提とした設計は成立しない。

第二に、OpenStack と Kubernetes のコントロールプレーンが同一ノードに同居する。本構成では 5 ノードに対して 3 つの quorum を配置する必要がある。OpenStack の Galera と RabbitMQ、platform クラスタの etcd、shared-tenant クラスタの etcd である。3 ノードの集合を 3 つ配置する場合、任意の 2 集合は必ず 1 ノード以上を共有するため、完全な分離はできない。共有を最小化する配置を選ぶことになる。

第三に、Ceph の recovery と backfill が OpenStack のコントロールプレーンと Kubernetes のワークロードと同じノードで実行される。CPU・メモリ・ディスク帯域の競合を Ceph 側のスケジューラとホストの cgroup の双方で制御する必要がある。

第四に、Galera・RabbitMQ・MON・RGW をどのノードに置くかが VM 配置の前提となる。この組み合わせを先に決める。

ネットワークの系統

Ceph 用のディスクネットワークが分離され jumbo で運用される。この jumbo は他の系統に及ばない。

ネットワーク	用途	MTU
Ceph cluster	OSD 間の replication / recovery / backfill	jumbo
Ceph public	RBD クライアント。Nova / Cinder / VM / cephfs-csi	確認を要する
サービス / API	OpenStack API、HAProxy VIP、Galera、RabbitMQ	確認を要する
プロバイダ (WAN 側)	外部接続と Floating IP	1500
テナントオーバーレイ (Geneve)	VM 間通信。Kubernetes のノード間通信が乗る	1462
Pod (CNI)	Pod 間通信	1412

設計上の帰結

第一に、Ceph の jumbo はテナントおよび Pod のトラフィックには及ばない。Geneve のトンネル端点は Ceph のディスクネットワークとは別の網に載るため、ディスクネットワークの MTU はテナント網の MTU 算定に関与しない。

第二に、Ceph の public network が 1500 である場合、VM からの RBD 入出力は 1500 で行われる。jumbo の効果は OSD 間の replication と recovery に限定され、ゲストのディスク性能には及ばない。全 VM の root を RBD に置く設計では、public network の MTU がコンテナイメージの取得時間、ひいては Pod の起動時間に影響する。cluster と public を分離しているか、双方が jumbo かを確認する。

第三に、スイッチが jumbo に対応していることは Ceph のディスクネットワークの存在から確認できる。したがって Geneve 用に別の jumbo VLAN を確保する選択肢が成立し得る。この場合 Pod の MTU を 1412 より大きく取れる。ただし NIC 本数とスイッチポートの余裕に依存するため、まずは 1500 の物理網を前提として構築する。

第四に、Ceph のディスクネットワークに Geneve を相乗りさせてはならない。recovery や backfill の最中にテナント通信が影響を受け、逆にテナントのバーストが Ceph のレイテンシを悪化させる。分離した目的が失われる。

MTU 設計

Geneve のヘッダ長が 38 バイトで確定しているため、各層の MTU は一意に定まる。

層	MTU	根拠
プロバイダ / サービス網 (物理)	1500	IPoE の標準値
テナント網 (Geneve)	1462	1500 から Geneve のヘッダ長 38 を引く
Pod (Cilium の VXLAN)	1412	1462 からカプセル化分 50 を引く
TCP セグメント最大長 (IPv4)	1372	1412 から IP と TCP のヘッダ 40 を引く
TCP セグメント最大長 (IPv6)	1352	1412 から IPv6 と TCP のヘッダ 60 を引く

経路上の最小 MTU は Pod の 1412 であり WAN の 1500 ではない

制約は内部のオーバーレイ側にある。Pod が送出したフレームは VXLAN で 1462、さらに Geneve で 1500 になって物理網を通る。この 1500 が物理網の上限と一致するため余裕はない。

一方 WAN へ出る際はカプセル化が外れるため Pod の 1412 がそのまま乗る。外部との通信で不足するのは WAN 側ではなく、外部から Pod へ向かう方向の内部区間である。

設定すべき箇所

Neutron には物理網の MTU として 1500 を、Geneve のヘッダ長として 38 を与え、テナント網に 1462 が適用されたことを確認する。CNI には MTU を 1412 で明示指定し、自動検出に任せない。自動検出で 1500 のままになると Pod は最大セグメント長 1460 を広告し、カプセル化後に物理網の上限を超えて破棄される。切り分けの難しい症状になる。

TCP は最大セグメント長の交渉で解決する。Pod は自身の MTU から 1372 を広告し、各方向の送信サイズは受信側が広告した値との小さい方で決まるため、双方 1372 以下に収まる。成立の条件は Pod の MTU が正しく設定されていることのみである。

BGP-GW では双方向の TCP の SYN に対して最大セグメント長を経路に合わせる処理を入れる。交渉が働けば冗長だが設定ミスへの保険になる。UDP と QUIC は調整が効かないため、ICMP の Fragmentation Needed と ICMPv6 の Packet Too Big の通過が前提となる。

認証系 — Authentik

コンポーネント	役割	依存先
authentik server	OIDC / SAML プロバイダと管理 UI	PostgreSQL、Redis
authentik worker	バックグラウンドタスクと送信処理	PostgreSQL、Redis
LDAP outpost	LDAP の bind を受け core の API へ転送する	authentik core への到達性
PostgreSQL	永続化。CloudNativePG で運用する	—
Redis	セッションとキャッシュ	—

依存の連鎖 — outpost の稼働は可用性を意味しない

SSH が SSSD を経由して LDAP outpost に問い合わせ、outpost が core の API を呼び、core が PostgreSQL を参照する。この連鎖のいずれかが停止すれば bind は失敗する。

LDAP outpost は core への API 接続に依存する別の Deployment である。したがって outpost が Pod として動作していても、core または PostgreSQL が停止すれば認証は成立しない。outpost の稼働状態を認証可用性の指標にしてはならない。

循環依存の所在

platform クラスタの apiserver が OIDC で authentik を参照し、その authentik が同一クラスタ上で動作する。したがって authentik が停止すると、それを復旧するための認証が得られないという循環が生じる。

復旧経路は kubeadm が構築時にコントロールプレーンノードへ配布するクライアント証明書である。これはクラスタの CA が署名したもので、OIDC 認証器の状態に依存しない。したがって新たに資格情報をを用意する必要はなく、要件はコントロールプレーンノードへ到達する SSH 経路が LDAP outpost を経由しないことである。

脅威モデルと残余リスク

カーネルエスケープを試みる利用者が存在する前提で構成する。免責を前提としない。

対策	守る境界	攻撃者に要求される能力	扱い
gVisor	システムコール境界。ホストカーネルの攻撃面	公開された脆弱性と実証コードで成立し得る	採用。全ユーザー Pod に既定適用
2 クラスタ分離	Kubernetes のコントロールプレーン境界。加えてゲストカーネルとハイパーバイザ	エスケープ後にさらに仮想化層の突破が必要	採用。到達範囲を絞る層
Pod Security Admission / NetworkPolicy / ResourceQuota	Kubernetes API とネットワーク	設定漏れ、権限昇格	採用。宣言的に自動適用する
SMT のスレッド分離	SMT 兄弟スレッド間のマイクロアーキテクチャ側チャンネル	共存の制御、高レート of 継続的な測定、ノイズ処理	不採用。残余リスクとして受容

受容した残余リスクの範囲

Kubernetes ノードと OpenStack のコントロールプレーンは同一物理ホスト上で動作する。したがってこの受容はテナント間だけでなく、テナントと管理系の境界にも及ぶ。クラスタ分離が得るのはカーネル境界とハイパーバイザ境界であり、ハードウェア境界ではない。

gVisor で守られない範囲

DaemonSet として動作する CNI エージェント、CSI のノードプラグイン、メトリクスエクスポート、およびシステムコールの互換性の問題で例外として扱うワークロードは、通常のランタイムで特権付きで動作する。共用クラスタ上のすべてが gVisor で動くわけではないため、クラスタ分離の層は省略できない。

受容の前提条件

ホストカーネルの投機実行に対する緩和が有効であり、microcode が最新であること。当該世代は一部の脆弱性にハードウェア緩和を持つが、残りは microcode とカーネルの緩和に依存する。構築時にホストの緩和状態を確認し、緩和されていない項目があれば受容の前提が崩れるものとして扱う。

2 クラスタ分離の根拠

カーネル境界は両構成で成立する。差分は共有物の有無と、分離の保証水準にある。

単一クラスタでノードプールのみを分離した場合に残る共有物

共有物	リスク	プール分離で解決するか
単一の etcd	全 namespace の Secret が同一のストアに同居する。認証基盤のデータベース資格情報と利用者の Secret が同じ場所に置かれる	不可
単一の apiserver	apiserver、admission、認可の脆弱性および設定ミスが管理系にも及ぶ	不可
kubelet の信頼境界	テナントノードの kubelet 資格情報から昇格が成立した場合、到達範囲がクラスタ全体になる	不可
クラスタ全体スコープのリソース	カスタムリソース定義、クラスタロール、Webhook を共有する。テナントの追加が管理系に影響する	不可

主たる根拠 — 共有物の排除

ノードプールの分離では上表のいずれも解決しない。単一クラスタでの管理系への露出はカーネル経由ではなく API 経由であり、テナントが到達するのは共有された etcd、apiserver、クラスタ全体スコープのリソースである。

両クラスタの etcd を分離することは Secret の同居を解消する唯一の手段である。Kubernetes には namespace 単位で暗号化キーを分ける仕組みがないため、単一クラスタで同等の分離を得る代替案は存在しない。

対価はコントロールプレーン 3 台分のリソースである。

副次的な効果 — 分離の保証水準

カーネル分離自体は単一クラスタでも成立するが、成立の根拠が異なる。

単一クラスタでは taint と許容設定、ノード選択、admission によって「テナント Pod が管理系ノードに配置される」経路を塞いでいる。設定漏れ、許容設定を与える権限の誤付与、admission の失敗時の挙動によって破れる設定依存の分離である。

2 クラスタではその経路自体が存在しない。テナントのマニフェストを受け付ける apiserver が管理系ノードを知らないため、構造的な分離になる。

ただし分離されるのはゲストカーネルまでである。両クラスタの VM は同一の物理ホスト上に載るため、ハイパーバイザとハードウェアは共有される。

ワークロード配置

#	コンポーネント	配置	要点
1	認証基盤	platform クラスタ	Authentik と PostgreSQL。Keystone とは OIDC のフェデレーションで連携する
2	管理 API	platform クラスタ	ステートレス。アプリケーション資格情報で Keystone を呼ぶ。テナントの払い出しを行う主体
3	管理ポータルフロント	platform クラスタ	静的配信。管理 API と同一の Gateway 配下に置く
4	Terraform 実行環境	platform クラスタ	常駐サービスではないため CI のランナーとして配置する。状態はオブジェクトストレージに置く
5	Kubernetes 提供	shared-tenant クラスタ	共用クラスタ上で namespace を分離する
6	メールサーバー	OpenStack VM	本資料の対象外。リソースの予約のみ行う
7	ログと監視	platform クラスタと外部の受信口	メトリクスとログの保管はクラスタ内。外部へ出るのは死活の通知とアラートの識別情報のみ

配置の原則

管理系と利用者系は同一クラスタに置かない。根拠はコントロールプレーン境界である。

常駐しないものは実行体のみを置く。Terraform は状態を外部に持つため、クラスタ上に常時稼働させる必要はない。

状態を持ちインターネットに面するものは VM とする。送信元アドレスの安定性と、共有ストレージの制約が理由である。

再取得できるデータはオブジェクトストレージへ退避させ、クラスタ内には短期分のみを保持する。メトリクスとログが該当する。

自身を監視するものには、外部に独立した検知点を設ける。監視スタックが停止したときに沈黙するため、内部の監視だけでは検知できない。

ストレージ配置方針

ローカル SSD は OS と Ceph の DB / WAL に用いる。ここからの逸脱は etcd のみとする。

対象	配置	根拠
全 VM の root と追加ディスク	Ceph RBD。replica 3	SSD の用途に含めない
etcd のデータディレクトリ	ローカル SSD 上のブロックストレージ	同期書き込みのレイテンシ。唯一の例外
読み書き単一の永続ボリューム	RBD	Cinder の CSI ドライバで払い出す
読み書き共有の永続ボリューム	CephFS のサブボリューム	提供の要否を決める必要がある
オブジェクト	RGW	Terraform の状態、メトリクスとログの長期保管、バックアップ

受容する劣化

コンテナランタイムの領域、すなわちイメージのキャッシュと書き込み可能レイヤが HDD 上の RBD に載る。Pod の起動に要する時間が増加する。

緩和手段はコンテナレジストリのプロキシキャッシュと、ゴールデンイメージへの事前焼き込みである。いずれも緩和であって解消ではない。この劣化を受容する。

共有ボリュームを CephFS で提供する場合の条件

Manila を導入せず CephFS の CSI ドライバを Ceph に直結させる方法があるが、この場合 Pod から Ceph の MON への到達性が生じる。namespace ごとにサブボリュームを分け、接続に用いる資格情報は管理系の namespace のみに配置する。利用者の namespace には配布しない。

etcd 設計

原則 — 冗長化されたストレージの上に quorum を持つデータベースを置かない

etcd の 1 回の書き込みは 3 つのメンバがそれぞれ同期書き込みを行い、その各々が Ceph 上で 3 つのレプリカに複製される。etcd 自体が既に 3 重化されているため、下位レイヤの冗長性は書き込みの増幅とネットワークの往復を追加するだけである。

項目	内容
配置	ローカル SSD を用いるブロックストレージのバックエンドを別に設け、etcd 用のボリュームのみをそこから払い出す
対象	両クラスタのコントロールプレーン各 3 台
ゲスト内の扱い	専用のファイルシステムとして独立させ、アクセス時刻の更新を無効にしてマウントする
可用性の担保	ボリュームの耐久性に依存しない。raft とスナップショットで担保する
ホスト障害時	メンバを除去して追加し直す。ボリュームの退避は行わない
禁止事項	書き込みバリアの無効化。PLP がないため電源喪失時にデータベースが破損する
SSD の競合	OS と Ceph の DB / WAL と同一デバイスを共有する。ホスト側で OSD の入出力優先度を下げて緩和する

設定すべき箇所

etcd には、選挙に関わる待機時間を既定より緩めた値を与える。SSD が Ceph の DB / WAL と競合するため、既定値ではリーダー選挙が頻発する。

定期的な圧縮を有効にする。共用クラスタでは多数の利用者が Pod の作成と削除を繰り返すため、圧縮を行わないとデータベースが膨張し書き込みのレイテンシが悪化する。あわせてデータベースサイズの上限を設ける。

スナップショットの定期取得を CronJob として構成し、オブジェクトストレージへ退避する。ボリュームのスナップショットは取得できない構成であるため、これが唯一のバックアップ経路になる。

監視項目として、書き込みログの同期書き込みに要した時間の高位パーセンタイルを取得する。この値が閾値を超え続ける場合、専用 NVMe の追加を検討する判断材料とする。¹²

OpenStack 側に入れる設定

対象	設定項目	要件
Nova	CPU とメモリの超過割当比	いずれも超過を許さない値に固定する。既定は大きく、あふれた VM が Ceph OSD のスレッドを奪う
Nova	ホスト予約メモリ	OpenStack のコントローラサービスと Ceph OSD の実測消費を反映する。未設定では物理メモリ全量を配布可能と判断される
Nova	VM が使用する CPU の集合	ホストと Ceph OSD 用のスレッドを除外する。除外範囲はホストのスレッド列挙順に従って決める
Nova	スケジューラのフィルタ	アグリゲートの追加仕様によるフィルタを有効にする。既定に含まれないため、有効化しないと配置制約が無視される
Nova	イメージの格納方式	既定の RBD を維持する
Cinder	バックエンドの構成	RBD に加えてローカル SSD を用いるバックエンドを有効にし、etcd 用の種別を定義してそこへ割り当てる
Neutron	物理網の MTU	Geneve を載せる物理網の実 MTU に合わせる
Neutron	Geneve のヘッダ長	既定のままとするか明示するかを決め、テナント網に適用された MTU を確認する
Octavia	プロバイダドライバ	OVN のドライバを有効にし、amphora 用のイメージ・管理網・証明書の準備を無効にする
Ceph	OSD のメモリ目標値	実際の常駐サイズを測定した結果に基づいて決める。設定値は上限ではない
Ceph	OSD のスケジューラプロファイル	クライアント入出力を優先する設定にし、recovery 中に VM が停止するのを防ぐ
各 API サービス	ワーカー数	コア数から自動算出されるため 28 コア環境では過大になる。明示的に絞る

配置制約の実装方針

Nova のサーバーグループはグループ間の制約を表現できない。表現できる制約を区別する。

制約	実装	サーバーグループ単独で可能か
管理系のコントロールプレーンを特定のノード群に限り、互いに別ホストへ	ホストアグリゲートとフレーバの追加仕様、および厳格な非同居制約	不可
ユーザー系のコントロールプレーンを別のノード群に限り、互いに別ホストへ	別のアグリゲートで同様に構成する	不可
同一役割の 2 台を互いに別ホストへ	厳格な非同居制約	可
ワーカーを全ノードに分散	緩やかな非同居制約	可
役割の異なるもの同士を別ノードに	採用しない。理由は下記	不可

ホストを直接指定しない

可用性ゾーンにホスト名を含めて指定する方法は、制約ではなく計算結果を記述することになる。前提が変わったときに追従せず、ホスト障害時も停止したホストを指し続ける。加えて管理者権限を要する。

配置は制約として記述し、どのホストに載るかはスケジューラに委ねる。ノードを追加した際に自動で分散するという利点もある。

厳格な非同居制約を用いる理由

コントロールプレーンには緩やかな制約を用いない。緩やかな制約では 2 つのメンバが同一ホストに載る可能性があり、その 1 台の障害で quorum を失う。3 ノードの候補に対して 3 メンバを配置する場合、厳格な制約でも必ず成立する。

役割の異なるもの同士は分離しない

2 レプリカ構成では、異なる役割を別ノードに分けても同時障害で停止するサービス数の期待値は変わらず、冗長性が落ちるノードの数は増える。加えてアグリゲートを小さく絞ると、1 台の障害時に厳格な非同居制約を満たすホストが残らず再作成できなくなる。制約を減らしてスケジューラの自由度を残す。

ロードバランサ

項目	内容
プロバイダ	OVN のドライバを用いる。amphora を使用しないため、テナント数に比例した VM の増加が起きない
L7 の扱い	ドライバが対応しないため、L7 の制御は Gateway API 側で完結させる
TLS の終端	ドライバが対応しない。証明書管理と Gateway で終端する
ヘルスチェック	接続確認のみに対応する。HTTP レベルの確認は Pod の readiness 検査に寄せる
同一網からの接続	VIP への接続は送信元変換を伴う。東西通信で VIP を経由させない
利用者への提供	ロードバランサ型のサービスを作成できないよう割当上限を 0 とする。公開は Gateway 経由のみ
所有者	Octavia を操作するのは管理系が所有する Gateway のみとする

クラウドコントローラに入れる設定

OpenStack のクラウドコントローラマネージャには、使用するプロバイダとして OVN のドライバを指定し、あわせて負荷分散アルゴリズムに OVN のドライバが対応する方式を明示する。この指定を省略すると既定のアルゴリズムが要求され、ロードバランサの作成が失敗する。

ヘルスマニタの有効化と、外部公開に用いるネットワークの識別子も同時に指定する。

管理系が独占する理由

OVN のドライバはロードバランサの定義を OVN のテーブルに展開する。エントリ数は各ノードのコントローラの処理負荷に直結するため、利用者が自由に作成できる構成では namespace 数に比例して負荷が増える。

作成を管理系の Gateway のみに限れば、この問題と利用者起因の障害の双方が発生しない。利用者への公開経路は Gateway のルート定義に限定する。

共用クラスター 必須の分離策

以下は例外なく全 namespace に適用する。宣言的に自動適用し、手作業では設定しない。

#	対策	目的
1	Pod Security Admission を restricted で強制	特権コンテナ、ホストのファイルシステム・ネットワーク・プロセス名前空間の使用を封じる
2	イメージの常時取得を強制する admission プラグイン	他テナントの非公開イメージをノードのキャッシュから読み出すことを防ぐ
3	NetworkPolicy の既定拒否。入力と出力の双方	namespace 間の到達性を切る。Kubernetes の既定は全開であるため必須
4	ResourceQuota と LimitRange	CPU とメモリに加え、Pod・Secret・Service の個数、およびロードバランサの割当上限を含める
5	権限は namespace に限定したロールのみ	ノード、永続ボリューム、カスタムリソース定義、クラスターロールに関する権限を渡さない
6	Secret の保存時暗号化	全利用者の Secret が単一の etcd に同居するため必須。鍵管理サービスまたは対称鍵で暗号化する
7	kubelet のプロセス数上限	大量のプロセス生成によって他テナントごとノードを停止させることを防ぐ

自動適用を前提とする理由と、割当上限の決め方

namespace ごとに割当上限、通信制御、権限、セキュリティ水準のラベルを手作業で適用する運用は、規模が増えれば必ず漏れる。漏れた一つが分離全体を無意味にするため、テナントを表すカスタムリソースから自動適用する仕組みを用いる。管理 API はテナントを一つ作るだけでよい。適用漏れはポリシーエンジンの検査結果として定期的に検出する。

割当上限の具体値は、利用者数と一人あたりの想定ワークロード、およびノードあたりの実効的な割当可能量から逆算して決める。ノード役割の兼任によりノード間の余力が非対称になるため、総量が確定してから配分する。

共用クラスター強化策

#	対策	内容
8	gVisor をランタイムクラスとして導入	カーネルエスケープの発生確率を下げる層。単独では完結せず、クラスタ分離と組み合わせて成立する。仮想化の入れ子を要する方式は当該世代では性能上の負担が大きいため採らない。ポリシーエンジンでユーザー Pod にランタイムクラスを強制注入し、互換性の問題が出るワークロードのみ例外として扱う
9	テナント管理のためのコントローラ	テナントを表すカスタムリソースから割当上限、通信制御、権限、セキュリティ水準のラベルを自動適用する。クラスタ全体スコープのリソースの列挙も制限され、利用者が参照できる namespace が自身の分に限られる
10	Gateway API を用いる	広く使われる Ingress コントローラには注釈経由の設定注入という既知の攻撃面があるため、全利用者に Ingress の作成権を渡す構成は採らない。Gateway API では管理系が Gateway を所有し、利用者はルート定義のみを作成、参照は明示的な許可を要する形で権限が分離されている
11	CNI に L7 と DNS を扱えるものを用いる	通信制御をドメイン名および L7 の条件で記述できる。プロキシの置換により転送規則の数が namespace 数に対して線形に増加しない。通信の可視化により事故の切り分けが速くなる

確認を要する事項

CNI がソケット層で行うサービス解決が gVisor のサンドボックス内から機能するかは実機で確認する。機能しない場合は当該ノードプールでその機能を無効化し、仮想イーサネット経路の負荷分散に切り替える。

Gateway API の実装は複数あり、CNI に統合されたものは追加コンポーネントを持ち込まない利点があるが機能範囲が狭い。互換性の例外を認める判断基準もあわせて策定する。

Kubernetes クラスタに入れる設定

対象	設定項目	要件
apiserver	認証	Authentik を発行者とする OIDC を構成する。あわせて証明書による認証を残し、認証基盤の停止時の経路とする
apiserver	保存時暗号化	Secret を暗号化する。共用クラスタでは全利用者の Secret が同一の etcd に同居するため必須
apiserver	admission プラグイン	イメージの常時取得を有効にする。ノードによる自己申告を制限するプラグインは無効にしない
apiserver	監査	テナント namespace への書き込みと権限変更を記録する
etcd	選挙と圧縮	選挙の待機時間を緩め、定期圧縮とサイズ上限を設定する
クラウドコントローラ	ロードバランサ	OVN のプロバイダと対応する負荷分散アルゴリズムを明示する
CSI	ストレージクラス	RBD を用いる既定のクラスを定義する。共有ボリュームを提供する場合は別に定義する
CNI	MTU とプロキシ置換	MTU を明示指定する。転送規則の増加を避けるためプロキシの置換を有効にする
ランタイムクラス	gVisor	定義し、ノードプールを限定する条件を付与する
ポリシーエンジン	強制と検査	ランタイムクラスの注入、可用性予算の検証、適用漏れの検出を担わせる
全 namespace	セキュリティ水準	restricted を強制する水準として設定する

ワーカーノード ① イメージ

ノードは cattle として扱う。更新は入れ替えで行い、稼働中のノードを変更しない。

要素	内容
ベース OS	アンダーレイと同系統に揃える
コンテナランタイム	cgroup の管理を systemd に委ねる設定まで済ませて焼き込む
gVisor のランタイム	仮想化の入れ子を要しない実行方式を選ぶ
Kubernetes のコンポーネント	バージョンを固定し、パッケージマネージャによる意図しない更新を止める
ゲストエージェント	正常な停止とスナップショットの静止点確保のために必要
時刻同期	etcd と TLS の前提。必須
ストレージ関連のユーティリティ	CSI が要求するファイルシステムとマウントの道具一式
カーネルパラメータとモジュール	③ に記載
焼き込まないもの	クラスタへの参加処理。トークンは起動時に発行される短命な値である

イメージの属性として設定すべき事項

ディスクのバスとして SCSI を、モデルとして仮想 SCSI を指定する。これは Cinder の CSI ドライバの動作条件である。仮想ブロックデバイスを用いた場合、ディスクのシリアル番号が短く切られてボリューム識別子を含むデバイスパスが生成されず、ボリュームの接続は成功するがマウントに失敗する。原因の判別が難しい障害である。この指定の副作用として、ゲスト内のディスク名が SCSI の命名になる。追加ディスクのマウント指定と etcd 用ボリュームのマウント指定はこの命名を前提とする。

乱数生成器を仮想デバイスとして与える。これがない場合、多数の TLS ハンドシェイクでエントロピーの取得が待たされ、Pod の起動が遅延する。

ゲストエージェントの利用を宣言し、ハングしたノードを自動的に再起動するウォッチドッグの動作を指定する。ノードは cattle であるため、応答しないものは再起動して差し支えない。

ワーカーノード ② プロビジョニング

項目	内容
作成の主体	Terraform。台数、フレーバ、イメージ、ネットワークポートを変数として持つ
配置	緩やかな非同居制約でノード間に分散させる。ホスト名は指定しない
初期化	設定情報の受け渡しに構成ドライブを用い、初回起動時にクラスタへ参加させる
変更検知の除外	参加用トークンとイメージ識別子の変更を差分として扱わない
更新方法	イメージを更新し、対象を置き換えるよう明示して1台ずつ作り直す

変更検知から除外する理由

クラスタへの参加に用いるトークンは短命であるため、適用のたびに値が変わる。これを差分として扱うと、Terraform は初期化用データが変更されたと判断し、稼働中のワーカーを全台再作成する。

イメージ識別子も同様に除外する。イメージの更新は自動的に反映させるのではなく、対象を明示して1台ずつ置き換える。排出、作成、古いノードオブジェクトの削除という順序を運用手順として定める。

セキュリティグループに設定すべき事項

ノード間は同一グループを参照する形で相互に全許可とし、クラスタ内の通信制御は Kubernetes の NetworkPolicy に委ねる。二重に制御すると障害の切り分けが困難になる。

外部からは apiserver のポートを運用者のネットワークに限定し、SSH を踏み台に限定する。

ICMP の Fragmentation Needed と ICMPv6 の Packet Too Big を全送信元から許可する。既定のセキュリティグループはこれを落とすため明示的な許可が必要であり、これを欠くと Path MTU Discovery が機能せず、TCP は境界での調整で救っても UDP と QUIC が到達しない。

ワーカーノード③ ゲスト OS の設定

コンテナランタイムの領域を独立させる

コンテナランタイムが用いる領域を、ルートファイルシステムとは別のディスクに分離する。イメージのキャッシュが膨張してルートを埋めると、kubelet がディスク圧迫を検知して全 Pod を退避させるため、この分離は必須である。

ファイルシステムはプロジェクト単位の容量制限を有効にして作成する。書き込み可能レイヤに上限を設ける可能性があり、後から有効化するには再作成を要するため、使用予定がなくても最初から有効にする。

マウント時にはアクセス時刻の更新を無効にする。イメージの展開で大量のファイル操作が発生するため、更新の抑止が効く。

カーネルパラメータとして設定すべき事項

対象	要件
ファイル監視の上限	kubelet、CNI エージェント、CSI のノードプラグイン、各 Pod の監視処理が資源を消費する。既定値では namespace 数の増加とともに枯渇し、「Pod が起動しない」「設定情報の更新が反映されない」という形で静かに現れる。停止しないため原因の特定に時間を要する。最重要項目
ファイルディスクリプタの上限	同様に既定値では不足する
IP 転送とブリッジのフィルタ	Pod 間の転送に必要。有効化する
逆経路フィルタ	オーバーレイの構成で戻り経路が非対称になる場合に妨げとなるため無効にする
接続追跡の上限	Service の数と接続数に応じて既定値では不足する
近隣キャッシュの閾値	Pod 数が多い環境では既定値で溢れる
プロセス数の上限	ノード全体としての上限を引き上げる

上記はすべてイメージに焼き込み、全ノードで一律に適用する。ノードごとの差異を作らない。

ワーカーノード ④ kubelet とクラスタ参加

設定項目	要件
cgroup ドライバ	コンテナランタイムと揃える。不一致は起動時に検出されず、後から資源制限が効かない形で現れる
システム予約とKubernetes予約	ゲスト内の kubelet、コンテナランタイム、CNI エージェント、CSI プラグイン、init システムの消費量を測定して設定する
退避の閾値	メモリと2種のディスク領域について、余裕が閾値を下回ったときに Pod を退避させる条件を設ける
Pod あたりのプロセス数上限	テナント分離の必須項目として設定する
ノードあたりの Pod 数上限	Pod のアドレス範囲とノードのメモリから算出する
イメージ取得の並列化	直列取得を無効にし、複数イメージの取得を並行させる
カーネル設定の保護	有効にする。kubelet が独自にカーネル設定を書き換えないようにする
クラウドプロバイダ	外部のクラウドコントローラを用いる指定にする
ノードの識別子	メタデータサービスから取得したインスタンスの識別子を所定の形式で与える
taint	ノードプールを表す taint を参加時に自己申告させる

外部のクラウドコントローラを用いる場合の副作用

この指定を行うと、参加直後のノードに未初期化を示す taint が自動的に付与され、クラウドコントローラがこれを除去するまでスケジュール対象にならない。

クラウドコントローラが停止している場合、新規のワーカーは正常な状態にならず、症状はノード側の問題として現れる。ワーカーを追加する前にクラウドコントローラの稼働を確認する手順を運用に含める。

ノードの識別子はクラウドコントローラがノードと仮想マシンを対応付ける鍵である。これが正しく設定されない場合、ノードは初期化されない。

システム予約を設定しない場合、kubelet は Pod がノードのメモリ全量を要求できると判断する。仮想マシン上のワーカーではホスト側に退避先がないため、メモリ不足でノードごとと停止する。

ワーカーノード ⑤ ランタイムとラベル

コンテナランタイムに設定すべき事項

既定のランタイムとして通常のを指定し、cgroup の管理を init システムに委ねる設定を有効にする。あわせて gVisor のランタイムを追加のハンドラとして登録する。

設定ファイルの構造はコンテナランタイムのメジャーバージョンによって異なる。付属のインストーラが生成する設定は旧来の構造になる場合があるため、イメージに焼き込む時点で内容を確認し、起動試験を通す。

ランタイムクラスの定義

gVisor を選択するためのランタイムクラスを定義し、対応するハンドラを指定する。あわせてスケジュール条件としてノードプールを限定する。これによりユーザー Pod が、DaemonSet や CSI が動作するプールに配置されることを防ぐ。

kubelet が自己申告できるラベルは限られる

種別	自己申告	設定方法
taint	可	クラスタ参加時の設定として指定する
ノードによる偽装が制限された名前空間のラベル	可	kubelet の起動引数として与える
トポロジを表すラベル	可	クラウドコントローラが付与する
独自の名前空間を持つラベル	不可	クラスタ参加後に構成管理から適用する

ノードによる自己申告を制限する仕組みの帰結

独自の名前空間を持つラベルを kubelet の起動引数に含めると、クラスタへの参加そのものが失敗する。ノードプールの識別には、偽装が制限された名前空間のラベルを用いる。この名前空間は許可されており、かつノードが他のプールを偽装できない保証も伴う。

ワーカーノード ⑥ 検証と入れ替え

クラスタ参加後に以下を確認する。いずれも省略すると後の障害の原因を特定できなくなる。

#	確認内容	確認の意味
1	未初期化の taint が除去され、ノードの識別子が設定されている	クラウドコントローラが正常に動作している
2	Pod の MTU が想定値であり、境界を越える通信が成立する	断片化不可の指定を付けた通信で上限を確認する
3	コンテナランタイムの領域が独立したディスク上にある	ルートファイルシステムの枯渇を防げている
4	gVisor で Pod が起動する	ランタイムの登録とランタイムクラスが機能している
5	カーネルパラメータが適用されている	イメージへの焼き込みが有効になっている
6	ボリュームの接続でデバイスパスが解決される	イメージの属性設定が正しい

入れ替え手順

対象ノードから Pod を排出し、参加用のトークンを新たに発行して対象を明示的に置き換え、古いノードオブジェクトを削除する。この 3 段階が更新手順を兼ねる。稼働中のノードに変更を加えることはしない。

置き換えでは同一の制約に従って新しいイメージのノードが作成され、古いノードのボリュームは破棄される。

排出が完了しない条件を先に潰す

利用者が、レプリカ数と同じ値の最小稼働数を指定した可用性予算を作成すると、排出が完了しない。共用クラスタでは一人の設定でノード更新全体が停止する。

ポリシーエンジンで、最小稼働数がレプリカ数と等しくなる指定を拒否するか、最大停止数による指定を強制する。これはノードを入れ替える運用の前提条件である。

認証基盤停止時の復旧経路

単一の認証基盤に依存するシステムを列挙し、各々にローカルの資格情報を定義する。

システム	平常時の認証	認証基盤停止時の経路
管理系クラスタ	OIDC	構築時にコントロールプレーンへ配布されるクライアント証明書
ユーザー系クラスタ	OIDC	同上
Keystone	OIDC フェデレーション	ローカルの管理者。フェデレーションの構成により到達可否が変わる
管理ダッシュボード	Keystone 経由	使用せずコマンドラインに退避する
構成管理	OIDC	ローカルの管理者と初期発行される資格情報
コンテナレジストリ	OIDC	ローカルの管理者
コード管理	OIDC	ローカルの管理者
可視化	OIDC	環境変数で与える管理者
踏み台・各ノード・ホスト	SSH	鍵認証のみ。ディレクトリサービスを経由させない
管理コントローラ	ローカル	認証基盤と統合しない

最下層の原則

管理コントローラからホストのローカルユーザーへ、そこからノードへ、最後に配布済みの証明書へと至る。この連鎖の最下層が認証基盤に依存しないことのみが本質である。平常運用の SSH を認証基盤と連携させる場合、復旧用のローカルユーザーを別に維持する。

証明書の期限管理

コントロールプレーンに配布されるクライアント証明書は自動更新されない。有効期限は限られており、クラスタの更新操作または明示的な更新操作でのみ延長される。残存期間を監視し、閾値で通知する。認証基盤が停止している最中に証明書も期限切れであれば復旧手段が失われる。あわせて、権限設定そのものが壊れた場合に用いる上位の証明書も台帳に含める。

監視設計

apiserver の生存は監視スタックの生存を意味しない。両者を別に検知する。

検知対象	手段	実行場所
監視スタック自身の沈黙	常時発火する通知の到着が途絶えたことを外部で検知する	外部に受信口のみ
WAN と経路	BGP セッションの状態と上流への到達性	外部
仮想ネットワークと外部アドレス	外部アドレスへの接続確認	外部
apiserver	生存確認エンドポイントへの接続	外部
OVN の内部状態	コントローラの接続数と処理のメトリクス	内部
オブジェクトストレージ	小さなオブジェクトの書き込み・読み出し・削除による能動確認	内部
メトリクス転送の滞留	送信済みの時刻と最新の時刻の差	内部
収集対象の欠落	収集できている対象の数が基準を下回ったことを検知する	内部

保管はクラスタ内に置く

メトリクスとログの集約および保管はすべて管理系クラスタ上で行う。外部に計算資源を置くと、可用性の根拠、費用、利用者のデータを外部に持ち出す扱いの整理が必要になる。

外部に必要なのは受信口ひとつである。送出されるのは死活の通知とアラートの識別情報のみで、メトリクスとログの本体はクラスタ内に留まる。データの外部持ち出しは発生しない。

保管期間はクラスタ内を短くし、それ以降はオブジェクトストレージへ退避させる。

死活通知の要件

常時発火する通知を一つ用意し、実際のアラートと同一の経路で外部の受信口へ送る。一定時間到着が途絶えたら外部側で検知する。これにより収集の停止、通知処理の停止、経路の断絶、ネットワークの分断をまとめて検知できる。

別の経路で送ると検査対象がずれるため、必ず実際のアラートと同じ経路を通す。

あわせて、集約側の複製数は監視系ノードの台数から決める。台数が少ない場合、直近の未退避分を失う可能性を受容するかどうかを明示的に決める。

構成管理の責務分界

層	担当	対象
基盤資源	Terraform	プロジェクトと利用者、割当上限、ネットワーク、ホストアグリゲート、オブジェクトストレージのバケット、レジストリのプロジェクト
テナント定義	Terraform	テナントを表すリソース、割当上限、通信制御の定義を生成し、リポジトリへ反映する。クラスタへの適用は行わない
収束	Argo CD	リポジトリ上の定義を両クラスタへ同期する
アドオン	Argo CD	CNI、CSI、クラウドコントローラ、Gateway、ランタイムクラス

Terraform を適用の主体にしない

Kubernetes のリソースを Terraform から直接適用すると、コントローラが状態を更新するたびに Terraform が差分を検出し、収束が終わらない。Terraform の責務はリポジトリに定義を置くまでとし、クラスタへの適用は構成管理ツールに委ねる。

ポータル経由の流れは、利用者の申請を管理 API が受けて変数を生成し変更要求を作成、継続的インテグレーションで差分を確認、承認または自動的な統合を経て、構成管理ツールが収束させる、という順序になる。

状態の保管

Terraform の状態はオブジェクトストレージの S3 互換エンドポイントに置く。Swift 向けのバックエンドは既に削除されているため使用しない。バケットのバージョニングを有効にし、状態が破損した場合に復旧できるようにする。

排他制御はオブジェクトストレージの条件付き書き込みに依存する機能を用いる。この機能への対応状況は構築後に実機で確認し、対応していない場合は継続的インテグレーション側で実行を直列化する形に切り替える。

PREREQUISITES TO BUILD

構築前に確定すべき事項

以下は設計の決定または実測を要する。順序は依存関係に従う。

#	事項	確定方法	影響範囲
1	Galera・RabbitMQ・MON・RGW を置くノードの組み合わせ	設計決定	ホストアグリゲートの割当、3つの quorum の配置
2	OpenStack 各サービスの実メモリ消費	構築後に役割別に測定	ノードあたりの割当可能量、VMのサイジング
3	Ceph OSD の常駐サイズ。平常時と再構築中の双方	再構築を意図的に発生させて最悪値を取得	同上。ノード障害時の再作成余力
4	Ceph の総容量	構成確定後に確認	全 VM を RBD に収容できるか
5	Ceph の public network の MTU	cluster network とは別に確認	ゲストのディスク性能、Pod の起動時間
6	ローカル SSD の同期書き込み性能	同期書き込みを強制した測定	etcd 配置の妥当性、専用 NVMe の追加要否
7	テナント網に 1462 が適用されていること	ネットワーク属性の確認	CNI に指定する MTU の妥当性
8	Geneve を jumbo 網に載せるかの判断	NIC 本数とスイッチポートの確認	Pod の MTU を 1412 より上げられるか
9	ホストの投機実行に対する緩和状態	ホストの脆弱性情報の確認	SMT の残余リスク受容の前提
10	オブジェクトストレージの条件付き書き込み対応	実機で確認	Terraform の排他制御方式
11	ローカル SSD 上のボリュームの同期書き込みレイテンシ	ゲスト内で測定	etcd 配置方式の妥当性
12	CNI のソケット層機能と gVisor の相互動作	実機で確認	当該機能の有効化

1 と 2 が前提となる

ノード役割の組み合わせと OpenStack のコントローラサービスの実メモリ消費が確定するまで、VM のサイジングとノード別の配置は決められない。ノード役割の兼任によりノード間の余力が非対称になるため、均等配分を前提とした計画は成立しない。この 2 点を最初に確定させる。